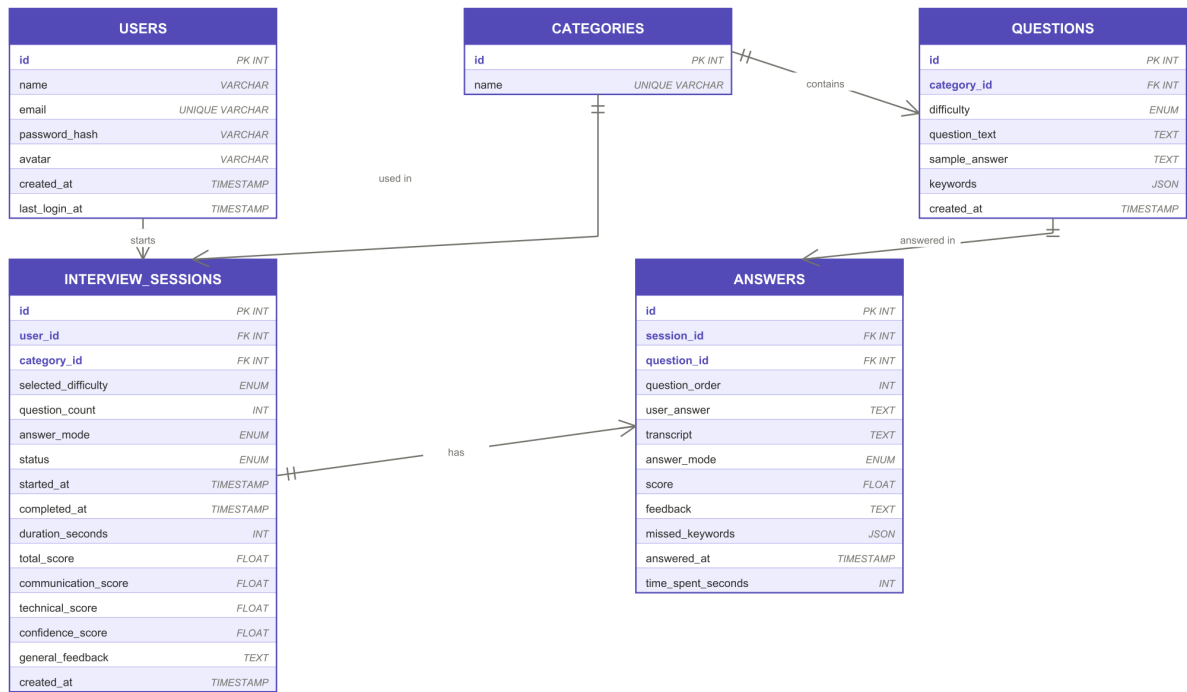


Smart Interview Trainer — Entity Relationship Diagram (v2)



Fields highlighted in the diagram: avatar, last_login_at (users) · selected_difficulty, communication_score, technical_score, confidence_score (interview_sessions) · question_order (answers).

1. Overview

The Smart Interview Trainer database is organized into five tables. The schema follows a **session-centric design**: a user configures a session (category, difficulty, answer mode, question count), the session generates a set of answers, and each answer ties back to a specific question. This structure supports both the core interview flow and longitudinal progress tracking via timestamps and scores.

Version 2 of this schema adds **last_login_at** to users for activity tracking, three analytical sub-scores (**communication_score**, **technical_score**, **confidence_score**) to interview_sessions for richer dashboard charts, and **question_order** to answers so the exact sequence of questions in a session can be reconstructed and analyzed.

2. Table Descriptions

users

Stores all registered accounts. The `avatar` field holds a reference (e.g. a filename or enum value) to the avatar the user selects in the UI. `email` is enforced as unique. `password_hash` stores a bcrypt or argon2 hash — never plaintext. `created_at` enables cohort analysis and account age tracking. `last_login_at` records the most recent successful login timestamp, useful for identifying inactive accounts, driving re-engagement logic, and displaying "last active" information in admin dashboards.

+ `last_login_at` **TIMESTAMP**

categories

A simple lookup table listing available interview domains — for example, "Frontend Development", "Behavioral", "System Design". The `name` column is unique to prevent duplicate categories. This table is intentionally lean; complexity lives in the questions, not the category.

questions

The content backbone of the app. Each question belongs to exactly one category (`category_id`) and has a difficulty level (e.g. beginner, intermediate, advanced). The `keywords` field stores a JSON array of important terms used by the feedback engine to evaluate answers. `sample_answer` provides a reference answer for scoring comparison.

interview_sessions

Records every session a user initiates. It captures the full configuration the user chose — `category_id`, `selected_difficulty`, `question_count`, `answer_mode` — so history is self-contained. `status` tracks lifecycle (`in_progress`, `completed`, `abandoned`). `started_at`, `completed_at`, and `duration_seconds` power the progress charts. `total_score` and `general_feedback` are written at session completion. Three new analytical sub-scores — `communication_score`, `technical_score`, and `confidence_score` — are stored separately so the dashboard can plot each dimension independently over time, giving users granular insight into which skill area needs improvement.

+ `communication_score` **FLOAT** · + `technical_score` **FLOAT** · + `confidence_score` **FLOAT**

answers

One row per question answered within a session. `question_order` records the exact position of this question in the session (1, 2, 3 ...) so the full interview sequence can be reconstructed for review or analysis — independently of how questions happen to be stored or sorted in the database. `user_answer` stores typed text; `transcript` stores the speech-to-text output in voice mode. `score` is a per-question numeric grade. `missed_keywords` is a JSON array identifying which expected keywords the user omitted — directly driving the detailed feedback UI. `time_spent_seconds` enables per-question analytics.

+ `question_order` **INT**

3. Relationships

Relationship	Cardinality	Description
<code>users → interview_sessions</code>	One-to-many	A user can start many sessions over time; each session belongs to exactly one user.
<code>categories → questions</code>	One-to-many	A category groups many questions; each question belongs to one category.
<code>categories → interview_sessions</code>	One-to-many	A session is scoped to one category; a category can appear in many sessions.
<code>interview_sessions → answers</code>	One-to-many	A session produces one answer row per question answered; all answers are scoped to their session. <code>question_order</code> on each answer row defines the sequence within that session.
<code>questions → answers</code>	One-to-many	The same question can be answered across many sessions by many users.

4. Design Decisions & Rationale

Why `last_login_at` is stored in `users`

Recording the most recent login timestamp serves three practical purposes. First, it enables re-engagement logic — the application can identify users who have not logged in for a defined period and trigger reminder emails or notifications. Second, it provides an "active users" metric for admin analytics without requiring a separate activity log table. Third, it can surface a "Welcome back — your last session was X days ago" message in the UI, which improves user experience at essentially zero schema cost.

Why `communication_score`, `technical_score`, and `confidence_score` are stored separately ♦ new

A single `total_score` is sufficient for a simple pass/fail summary, but it collapses three distinct performance dimensions into one number, making it impossible to plot meaningful progress charts per skill area. By storing the three sub-scores as separate FLOAT columns on `interview_sessions`, the dashboard can render a radar chart or three independent line charts — one for communication, one for technical accuracy, one for confidence — each plotted over time. This lets a user see, for example, that their technical score has improved steadily while their communication score remains flat, which is exactly the kind of actionable insight the app is designed to provide. Keeping them in the sessions table (rather than a separate scores table) avoids an extra join for every dashboard query and is appropriate at this project scale.

Why `question_order` is stored in `answers` ♦ new

The order in which questions are presented to a user during a session is not implied by any other field in the schema. Without `question_order`, there is no reliable way to reconstruct the original interview sequence — `answered_at` timestamps can overlap or be affected by network latency, and the insertion order of rows in a relational database is not guaranteed to reflect presentation order. Storing an explicit integer (1, 2, 3 ...) on each answer row makes the session fully reconstructable for review screens, exportable as a transcript, and correctly orderable for sequential analysis such as "did the user improve across the session?".

Why difficulty appears in both questions and interview_sessions

In questions, difficulty is an intrinsic property of the content (an advanced question stays advanced regardless of context). In interview_sessions, it is stored as selected_difficulty and records the user's chosen filter for that session — necessary for accurate progress charts (e.g. "show me only my advanced-level sessions over time").

Why category_id appears in both questions and interview_sessions

Questions have a permanent category. Sessions also store category so a user's history remains readable without joining through questions — especially useful when questions are later deleted or updated.

Why answer_mode is stored in both interview_sessions and answers

The session-level field captures the user's global choice (text vs voice). The answer-level field allows flexibility — a future version could let users switch mode mid-session, and analytics can be run at the individual answer level without depending on the session.

Why keywords and missed_keywords are JSON

The number of keywords per question varies. Storing them as JSON arrays avoids a separate junction table, which would add complexity without meaningful benefit at this project scale. For a production system at larger scale, a normalized keywords table would improve query performance and enable keyword reuse across questions.

5. Recommendations (without overcomplicating)

These are small, optional improvements worth considering:

- **Add a composite index on interview_sessions(user_id, created_at)** — the most common dashboard query pattern is all sessions for a specific user ordered by date. Without this index it becomes a full table scan as data grows.
- **Index answers(session_id, question_order)** — any query that reconstructs a session's question sequence will filter by session_id and sort by question_order. A composite index on these two columns makes that lookup efficient.
- **Consider a session_questions junction table** if you want to pre-assign questions before the user starts answering. Currently the set of questions in a session is implied by the answers table, so you cannot distinguish 'not yet answered' from 'not assigned'.
- **status, answer_mode, difficulty, and selected_difficulty should be database-level ENUMs** — restricting these to known values at the schema level prevents invalid data entering the database if the application layer is bypassed.
- **Add soft-delete to questions (is_active BOOLEAN)** — hard-deleting a question creates orphaned foreign keys in the answers table. A soft-delete flag preserves historical session data integrity.
- **Store communication_score, technical_score, and confidence_score as NOT NULL with a DEFAULT of 0** — this prevents NULL handling complexity in dashboard aggregation queries and makes the column semantics explicit: a score of 0 means the session was not evaluated, not that data is missing.